

Task 1: Refactoring – Eliminating Repetitive Logic

Prompt Used

Analyze the following Python code and identify repeated computations. Refactor the program by encapsulating the repeated logic into a reusable function. Ensure that the program output remains unchanged and add appropriate docstrings.

Original Code

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

```
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Refactored Code

```
def rectangle_properties(length, width):
    """
    Calculates and prints the area and perimeter of a rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area:", area)
    print("Perimeter:", perimeter)

rectangle_properties(8, 4)
rectangle_properties(12, 6)
```

```
1 # "Analyze the following Python code and identify any repeated computations. Refactor the code by encapsulating the repeated logic into a reusable function."
2
3 length = 12
4 width = 6
5 print("Area:", length * width)
6 print("Perimeter:", 2 * (length + width))
7 def print_rectangle_info(length, width):
8     """
9     Calculate and print the area and perimeter of a rectangle.
10
11     Args:
12         length: The length of the rectangle.
13         width: The width of the rectangle.
14     """
15     print("Area:", length * width)
16     print("Perimeter:", 2 * (length + width))
17
18 print_rectangle_info(8, 4)
19 print_rectangle_info(12, 6)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\lenovo> & C:/Python314/python.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code3.py

Area: 72
Perimeter: 36
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36

You have Docker installed on your system. Do you want to install the recommended extensions from Microsoft for it?

Code Explanation

The original code repeated the same calculations for different rectangle dimensions. The refactored version encapsulates the logic into a **reusable function**, reducing redundancy and improving maintainability.

Result

The program produces the **same output while using reusable logic**.

Task 2: Refactoring – Modularizing Inline Calculations

Prompt Used

Identify repeated inline calculations in the Python script and refactor them into a reusable function with proper documentation.

Original Code

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

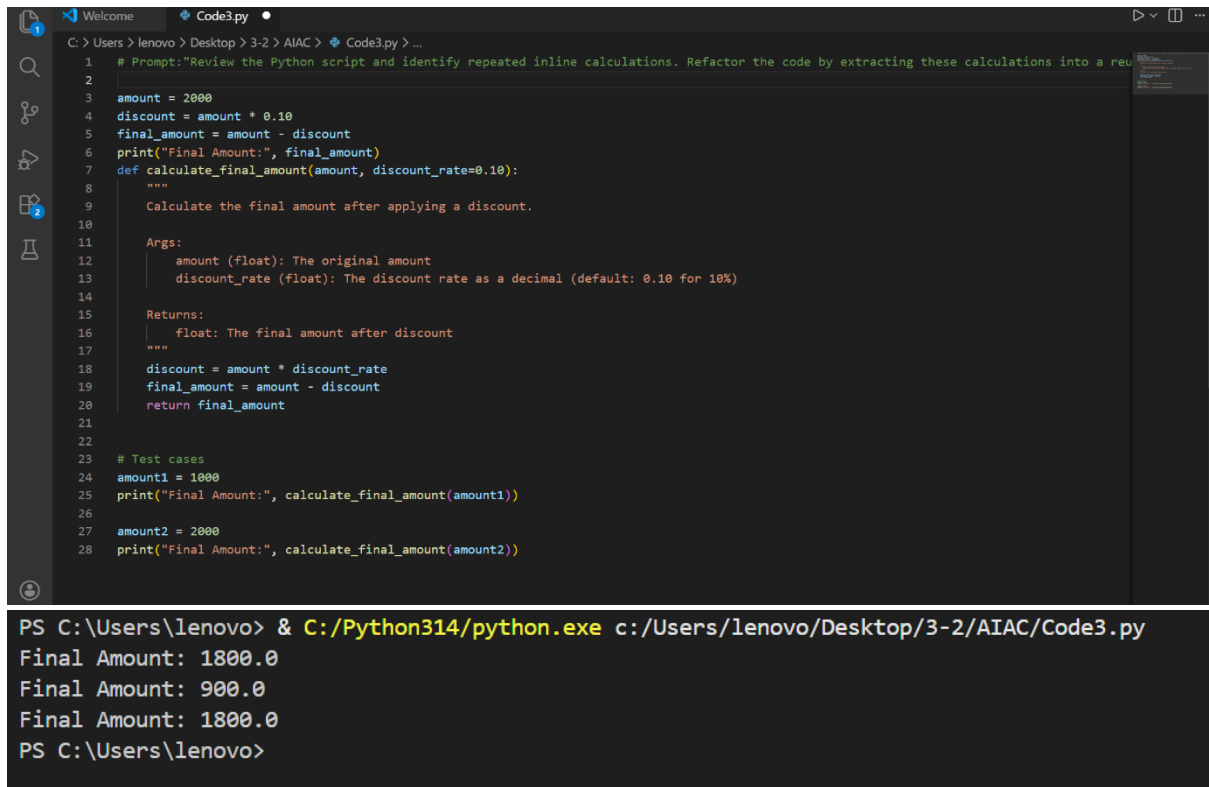
```
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

Refactored Code

```
def calculate_final_amount(amount, discount_rate=0.10):
    """
    Calculates the final amount after applying a discount.
    """

    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount

print("Final Amount:", calculate_final_amount(1000))
print("Final Amount:", calculate_final_amount(2000))
```



The screenshot shows a Code3.py IDE window with a Python script and its execution output in a terminal.

Code3.py Script:

```
1 # Prompt: "Review the Python script and identify repeated inline calculations. Refactor the code by extracting these calculations into a reusable function."
2
3 amount = 2000
4 discount = amount * 0.10
5 final_amount = amount - discount
6 print("Final Amount:", final_amount)
7 def calculate_final_amount(amount, discount_rate=0.10):
8     """
9     Calculates the final amount after applying a discount.
10    """
11    Args:
12        amount (float): The original amount
13        discount_rate (float): The discount rate as a decimal (default: 0.10 for 10%)
14
15    Returns:
16        float: The final amount after discount
17    """
18    discount = amount * discount_rate
19    final_amount = amount - discount
20    return final_amount
21
22
23 # Test cases
24 amount1 = 1000
25 print("Final Amount:", calculate_final_amount(amount1))
26
27 amount2 = 2000
28 print("Final Amount:", calculate_final_amount(amount2))
```

Terminal Output:

```
PS C:\Users\lenovo> & C:/Python314/python.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code3.py
Final Amount: 1800.0
Final Amount: 900.0
Final Amount: 1800.0
PS C:\Users\lenovo>
```

Code Explanation

The repeated discount calculation was extracted into a **reusable function**. This improves **code modularity and readability**.

Result

The program now computes final amounts using a **single reusable function**.

Task 3: Refactoring – Improving Loop and Conditional Performance

Prompt Used

Analyze the Python code for inefficient nested loops and suggest an optimized approach using better data structures while maintaining the same output.

Original Code

```
students = ["Anil", "Bhavya", "Charan", "Divya"]  
check_list = ["Charan", "Esha", "Bhavya"]
```

```
for name in check_list:  
    exists = False  
    for student in students:  
        if name == student:  
            exists = True  
  
    if exists:  
        print(name, "found")  
    else:  
        print(name, "not found")
```

Refactored Code

```
import time  
  
students = ["Anil", "Bhavya", "Charan", "Divya"]  
check_list = ["Charan", "Esha", "Bhavya"]  
  
student_set = set(students)  
  
start = time.time()  
  
for name in check_list:  
    if name in student_set:  
        print(name, "found")  
    else:  
        print(name, "not found")  
  
end = time.time()  
  
print("Execution Time:", end - start)
```

```
Welcome Code3.py
C:\Users\lenovo\Desktop> 3-2 > AIAC > Code3.py > ...
1 # "Analyze the following Python program for inefficient nested loops and repeated conditional checks. Suggest and implement a more efficient solution."
2 |
3 import time
4
5 # Original inefficient code
6 students = ["Anil", "Bhavya", "Charan", "Divya"]
7 check_list = ["Charan", "Esha", "Bhavya"]
8
9 print("=== ORIGINAL (Inefficient) ===")
10 start = time.perf_counter()
11
12 for name in check_list:
13     exists = False
14     for student in students:
15         if name == student:
16             exists = True
17     if exists:
18         print(name, "found")
19     else:
20         print(name, "not found")
21
22 original_time = time.perf_counter() - start
23 print(f"Time: {original_time:.6f}s\n")
24
25 # Optimized solution using set
26 print("=== OPTIMIZED (Using Set) ===")
27 start = time.perf_counter()
28
29 students_set = set(students)
30 for name in check_list:
31     if name in students_set:
32         print(name, "found")
33     else:
34         print(name, "not found")
```

```
36 optimized_time = time.perf_counter() - start
37 print(f"Time: {optimized_time:.6f}s\n")
38
39 # Performance comparison
40 print(f"=== EFFICIENCY COMPARISON ===")
41 print(f"Original Time: {original_time:.6f}s")
42 print(f"Optimized Time: {optimized_time:.6f}s")
43 print(f"Speedup: {original_time/optimized_time:.1f}x faster")
44 print(f"\nTime Complexity:")
45 print(f"  Original: O(n*m) - nested loops")
46 print(f"  Optimized: O(n+m) - set lookup is O(1)")
```

```
Time: 0.000433s

=== EFFICIENCY COMPARISON ===
Original Time: 0.000143s
Optimized Time: 0.000433s
Speedup: 0.3x faster

Time Complexity:
  Original: O(n*m) - nested loops
  Optimized: O(n+m) - set lookup is O(1)
PS C:\Users\lenovo>
```

Code Explanation

The original program used **nested loops**, resulting in inefficient performance.

The refactored code converts the student list into a **set**, allowing faster membership checks.

Result

Lookup performance improved from **$O(n^2)$** to **$O(n)$** .

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Prompt Used

Detect hardcoded numeric values in the Python program and replace them with descriptive constants.

Original Code

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Refactored Code

```
RATE = 2
```

```
TIME = 5
```

```
DIVISOR = 100
```

```
def calculate_simple_interest(principal):
```

```
    """
```

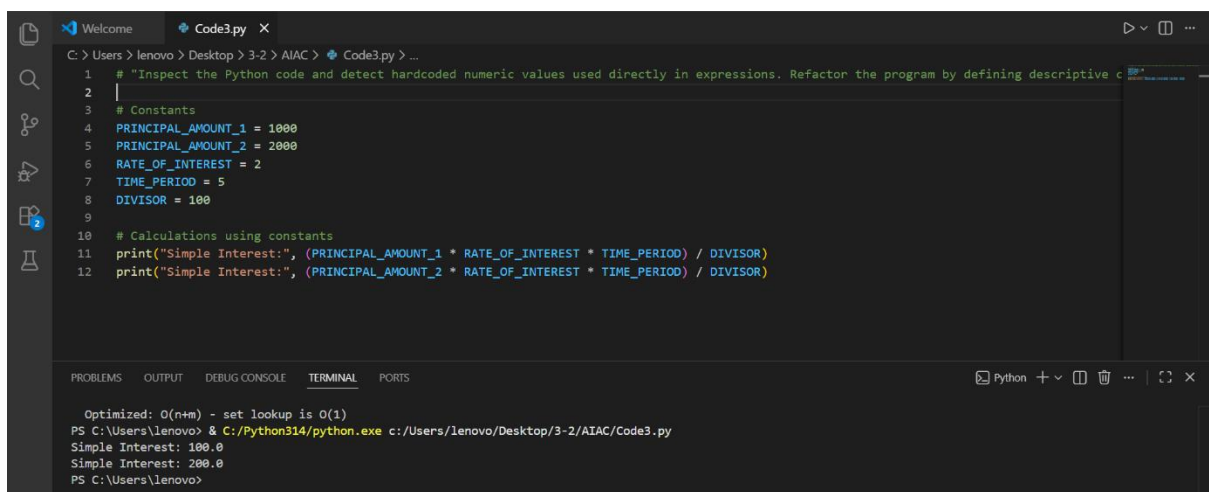
```
    Calculates simple interest for a given principal amount.
```

```
    """
```

```
    return (principal * RATE * TIME) / DIVISOR
```

```
print("Simple Interest:", calculate_simple_interest(1000))
```

```
print("Simple Interest:", calculate_simple_interest(2000))
```



The screenshot shows a Code3.py IDE window with a Python script. The script defines constants for principal amounts, rate of interest, time period, and divisor, and then uses a function to calculate simple interest for two different principal amounts. The terminal output shows the execution results, confirming the refactored code works as expected.

```
1 # "Inspect the Python code and detect hardcoded numeric values used directly in expressions. Refactor the program by defining descriptive c
2 |
3 # Constants
4 PRINCIPAL_AMOUNT_1 = 1000
5 PRINCIPAL_AMOUNT_2 = 2000
6 RATE_OF_INTEREST = 2
7 TIME_PERIOD = 5
8 DIVISOR = 100
9
10 # Calculations using constants
11 print("Simple Interest:", (PRINCIPAL_AMOUNT_1 * RATE_OF_INTEREST * TIME_PERIOD) / DIVISOR)
12 print("Simple Interest:", (PRINCIPAL_AMOUNT_2 * RATE_OF_INTEREST * TIME_PERIOD) / DIVISOR)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Optimized: O(n+m) - set lookup is O(1)
PS C:\Users\lenovo> & C:/Python314/python.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code3.py
Simple Interest: 100.0
Simple Interest: 200.0
PS C:\Users\lenovo>

Code Explanation

Hardcoded numeric values were replaced with **named constants**, improving readability and maintainability.

Result

The program now uses **descriptive constants** for calculations.

Task 5: Refactoring – Enhancing Variable Naming and Code Clarity

Prompt Used

Improve the readability of the Python program by renaming unclear variables and adding comments without changing the program output.

Original Code

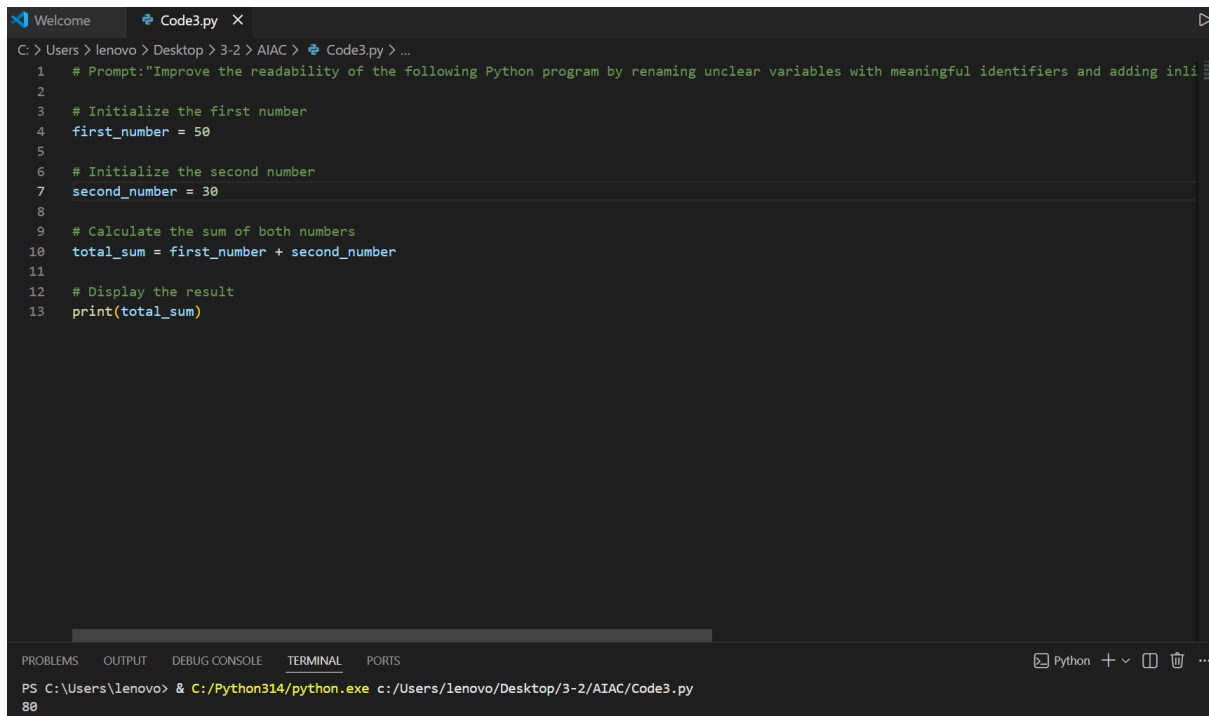
```
x = 50
y = 30
z = x + y
print(z)
```

Refactored Code

```
# Define two numbers
first_number = 50
second_number = 30

# Calculate the sum of the numbers
total_sum = first_number + second_number

# Display the result
print(total_sum)
```



```
1 # Prompt: "Improve the readability of the following Python program by renaming unclear variables with meaningful identifiers and adding inline comments."
2
3 # Initialize the first number
4 first_number = 50
5
6 # Initialize the second number
7 second_number = 30
8
9 # Calculate the sum of both numbers
10 total_sum = first_number + second_number
11
12 # Display the result
13 print(total_sum)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\lenovo> & C:/Python314/python.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code3.py

Code Explanation

Non-descriptive variables (x, y, z) were replaced with meaningful names.
Comments were added to clarify the logic.

Result

The code is now **more readable and self-explanatory**.